Day 2 | Build connected, intelligent Azure applications

From Azure fundamentals to messaging, practical integration and Microsoft AI

Understand → Create → Demonstrate → Explain

Experienced developers • Azure beginners • Fictional building-society scenario

Today’s learning journey

09:45–17:30 • Instructor demonstration followed by guided practice

09:45	Day 1 recap and architecture
10:25	Azure Service Bus: why, what and how
11:45	Queue lab + short SQL / Functions demo
13:30	Microsoft Azure AI: services and selection
14:30	Azure AI practical walkthrough
16:00	Real-world scenario and design discussion
17:00	Knowledge check and safe cleanup

Day 1 recap | Azure foundations, identity, compute and monitoring

A quick refresher: the Azure services and concepts already introduced

Cloud service models

Pizza analogy: IaaS, PaaS, SaaS

Azure organisation

Entra tenant: identity | Management group → subscription → RG → resource

Identity

Managed identity, app registration, service principal and enterprise application

Compute and orchestration

Azure Functions and Logic Apps

Data and testing

Blob Storage and Azure Load Testing

Observability

Log Analytics workspace and Application Insights

The pizza analogy: who manages what?

A useful way to distinguish IaaS, PaaS and SaaS

On-premises

You manage almost everything

IaaS

Provider runs infrastructure; you manage OS and application

PaaS

Provider manages platform; you focus on application and data

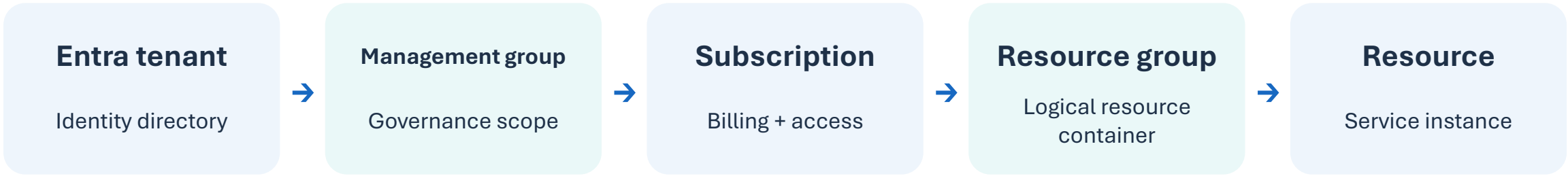
SaaS

Provider operates the complete application

Ask: Where do Azure Functions, Azure Virtual Machines and Microsoft 365 fit?

Azure hierarchy: know where you are working

Tenant = identity boundary; management groups organise subscriptions



The arrows are conceptual: the tenant is not an Azure RBAC parent scope.

Identity: four terms developers must distinguish

Do not confuse the app definition, its identity or permissions

App registration

Defines an application in Microsoft Entra ID.

Service principal

Tenant-local security identity for an application.

Enterprise application

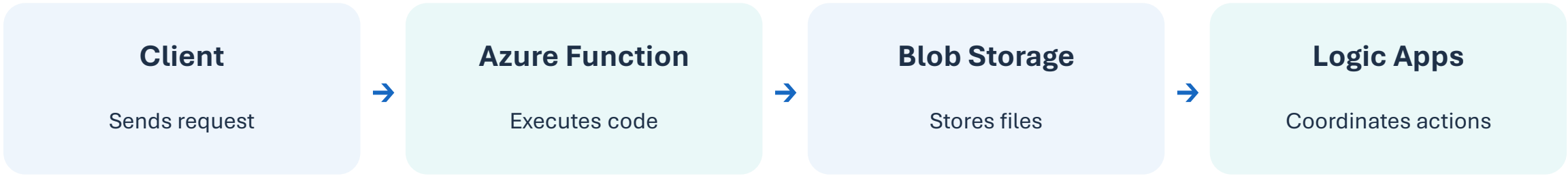
Portal representation of a service principal.

Managed identity

Azure-managed identity for workloads; avoids storing credentials.

Day 1 recap | Functions, Logic Apps, Storage, Load Testing and monitoring

A possible application flow—not a claim that every integration was deployed



Application Insights + Log Analytics help investigate performance and failures.

Five quick questions before we move on

Discuss in pairs for three minutes

- Why choose a Function rather than manage a virtual machine?
- How is a managed identity different from a stored client secret?
- What does a Logic App help orchestrate?
- Where would you store an uploaded document?
- How would you investigate a slow or failing request?

From Azure basics to a connected application

FOUNDATIONS & IDENTITY

- Pizza analogy: IaaS / PaaS / SaaS
- Tenant → subscription → resource group → resource
- App registration → service principal → enterprise app
- Managed identity + scoped permissions

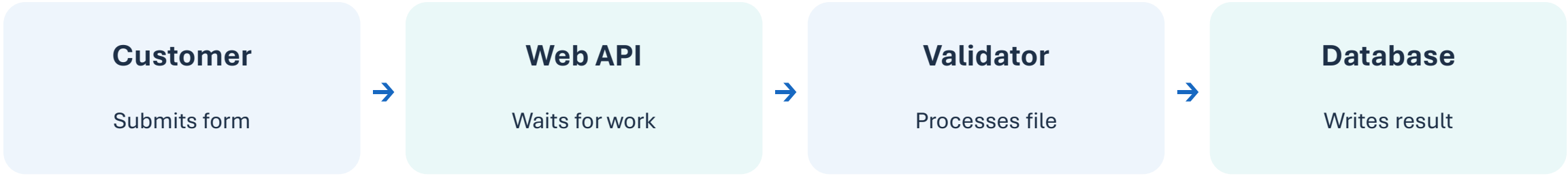
APPLICATION & OPERATIONS

- Azure Functions: event-driven code
- Logic Apps: workflow orchestration
- Blob Storage: unstructured files
- Azure Load Testing: performance evidence
- Application Insights + Log Analytics: telemetry / KQL

TODAY: Service Bus demo is deployed. AI enrichment is a separate walkthrough / future extension.

A mortgage application takes too long

Imagine every request waits for document validation and downstream processing



What happens when 500 applications arrive at once—or validation is temporarily unavailable?

What is Azure Service Bus?

A fully managed enterprise message broker for reliable asynchronous communication

Producer

The application sends a small message describing work to be done. It does not need to wait for processing.

Consumer

A separate worker receives the message, processes it and settles it after successful handling.

A queue is a buffer between the two components.

Why use a message queue?

Compare tightly coupled and decoupled application designs

Without a queue

The API waits for downstream work. Slow processing delays the user; a dependency outage can cause request failures.

With Service Bus

The API accepts work and sends a message. A worker processes it later; queued messages help absorb bursts.

Important: message accepted ≠ business process completed.

Core concepts: namespace, queue, topic

Choose the pattern and tier: topics/subscriptions require Standard or Premium

Namespace

Top-level container for Service Bus messaging entities.

Queue

Competing consumers share work; a message may be redelivered.

Topic + subscription

Publish once; independent subscriptions can receive copies.

Dead-letter queue

Holds messages requiring investigation after delivery/processing problems.

Queue architecture for a building society

Send only a job identifier and non-sensitive reference

Web app

Accepts application

→

Service Bus queue

Stores work message

→

Worker / Function

Processes asynchronously

→

Status store

Tracks outcome

Example message: { "applicationId": "DEMO-1042", "action": "validate" }

Delivery semantics: what can go wrong?

Reliable messaging still requires careful application design

At-least-once delivery

A message may be delivered again after a failure or expired lock.

Peek vs receive

Peek inspects without removing. Receive obtains a message for processing.

Complete / abandon

In PeekLock, complete after success; abandon or let lock expire to retry.

Idempotency

Use message/job IDs so duplicate delivery does not duplicate business actions.

Create a Service Bus namespace

Instructor-led Azure Portal demo • use only an approved subscription

- Search for Service Bus → Create.
- Select subscription, approved resource group and region.
- Enter a globally unique namespace name.
- Choose the approved pricing tier; check features and cost.
- Review + create, then open the namespace.

Permissions or policy blocked? Stop and ask the resource owner—do not bypass controls.

Create a queue and configure it

Namespace → Queues → + Queue

1 Name

Use an approved disposable training queue.

2 Settings

Review max size, TTL and lock duration. Duplicate detection requires a supported tier.

3 Access

Use approved Microsoft Entra data-plane RBAC where configured.

4 Verify

Open queue overview and confirm it is empty and isolated.

LIVE DEMO

Send → Peek → Receive → Complete

Service Bus Explorer • dedicated queue with no competing consumer

Send

Add fictional JSON

→

Peek

Inspect message

→

Receive

PeekLock mode

→

Complete

Settle after success

Success criteria: identify your message, explain its state and verify the chosen receive mode.

Your turn: test a message safely

10–15 minutes • record the evidence, not secrets

- Open your assigned namespace and isolated queue.
- Send one fictional application reference and note its MessageId.
- Peek and locate the message without removing it.
- Receive using the approved mode and settle your own message.
- Explain why sending does not prove processing succeeded.

Two versions: do not confuse the demos

Use fictional UTF-8 text only; neither version makes financial decisions

Original local starter

In-memory store and worker. Port 5193. Data is not durable across restarts.

Separate Azure version

Local Razor UI on 5293; real SQL, Service Bus and hosted .NET 10 Functions.

What Completed means

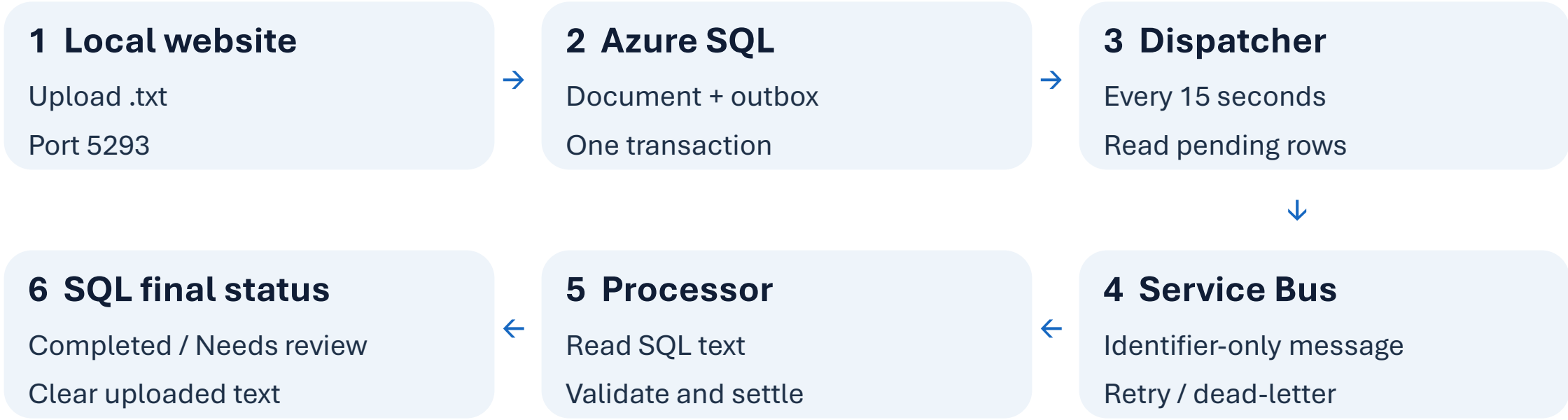
The sample passed deterministic format checks. No OCR, AI or identity verification.

What stays local

The website has no sign-in and is loopback-only. Do not publish it unchanged.

Deployed demo: SQL outbox → queue → Function

The website accepts work; a later SQL update records the processing outcome



Send first, then mark dispatched. Redelivery is possible; finalization is idempotent.

Three checks: network, identity and permission

Managed identity removes stored credentials; it does not bypass a firewall

Network: can I connect?

SQL firewall/public access permits or rejects the connection path.

Identity: who am I?

Hosted Functions use system-assigned managed identity and Entra tokens.

SQL authorization

A contained external user plus table GRANTs permits SELECT / UPDATE.

Service Bus authorization

Queue-scoped Data Sender and Data Receiver allow message operations.

A valid identity can still fail network access. An open network does not grant data access.

Instructor demo: one upload, end-to-end evidence

Use the Azure-backed website at http://127.0.0.1:5293

1 Prepare

Confirm account, firstdb, queue, identity and both hosted Functions. Use fictional samples only.

2 Upload

Sample 01 + LBS-DEMO-1001. Capture the document GUID from the details URL.

3 Observe

Refresh Queued → Completed. Show processor telemetry, not just timer success.

4 Explain

SQL stores the job; Service Bus delivers the identifier; Function finalizes SQL and clears text.

Observed result: new uploads completed

Owner-supplied evidence | 29 October 2026, around 22:05 UTC

LBS-DEMO-1001	Completed	2026-10-29 22:05:21	View status
LBS-DEMO-1001	Completed	2026-10-29 22:04:40	View status
LBS-DEMO-1001	Completed	2026-10-29 22:03:50	View status
LBS-DEMO-1001	Completed	2026-10-29 22:01:42	View status

Four successful sample-01 jobs.

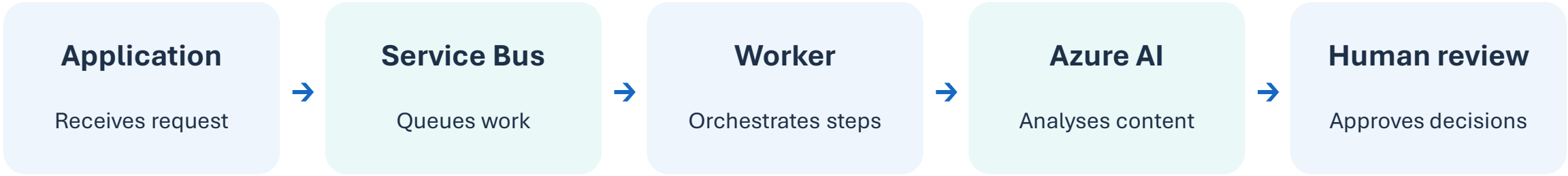
Evidence of these runs, not a new ten-case regression or production certification.

Old Queued rows can remain if their messages were already dead-lettered. Recovery is separate.

TRANSITION

From messaging to intelligent processing

Messaging moves work; AI can help interpret or classify its content



AI assists the workflow; identity, data protection, validation and human oversight still matter.

What do we mean by Azure AI services?

Select services based on the problem—not because AI is available

Azure AI Document Intelligence

Extract structured fields from forms and documents.

Azure AI Language

Analyse text, classify content or detect entities.

Azure AI Search

Index and retrieve relevant enterprise information.

Microsoft Foundry / Azure OpenAI

Build generative AI experiences using deployed models and governed access.

Which AI service should we choose?

Map the business requirement to the appropriate capability

Read a mortgage PDF	→ Document Intelligence
Find answers in approved policies	→ Azure AI Search + a language model
Summarise an applicant's note	→ Generative model with safeguards
Flag ambiguous extraction	→ Confidence checks + human review

Create and test an Azure AI resource

Demo route: document extraction using fictional sample documents

- Confirm approved subscription, region, resource and access.
- Create or open the approved Document Intelligence resource.
- Open Document Intelligence Studio or supported portal experience.
- Choose Read or Layout; upload the supplied fictional PDF.
- Run analysis; inspect extracted fields, confidence and limitations.

What should learners observe?

AI output is evidence to review—not an automatic business approval

Model-specific results

Read: text. Layout: structure and tables. Semantic fields depend on the model/features.

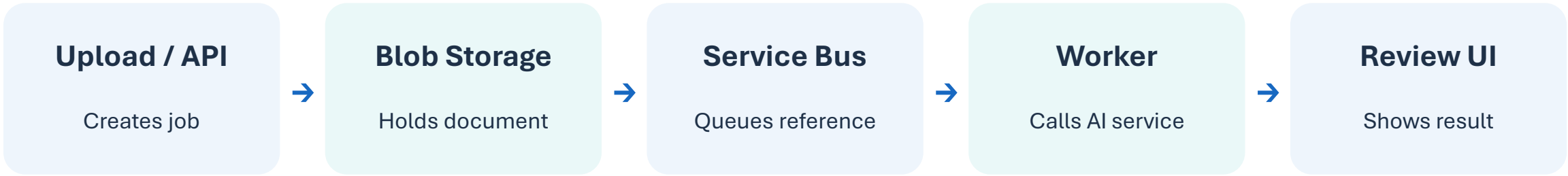
Important limitations

Model mistakes, missing fields, confidence uncertainty, sensitive data and operational cost.

Question: Which fields would require human confirmation before a lending decision?

Putting Service Bus and Azure AI together

Illustrative target architecture—not an already deployed integration



Production design also needs Entra identities, durable status, retries, telemetry and access controls.

How do we know the workflow is healthy?

Bring yesterday's monitoring services into the design

Application Insights

Request traces, failures, dependencies and latency.

Log Analytics

Query correlated operational logs using KQL.

Azure Load Testing

Test an approved workload under controlled traffic.

Cost Management

Review spend, forecasts, budgets and resource-level cost drivers.

Explain the architecture without the slides

Use the fictional mortgage-processing scenario

- Why is Service Bus useful when downstream processing is slow?
- What is the difference between a queue and a topic?
- Does Peek remove a message? What does Complete do?
- Which Azure AI service would extract fields from a form?
- Where would you look for failures, latency and costs?

What we learned today

From cloud fundamentals to a practical, governed integration design

Recall

Azure hierarchy, identity, compute, storage and monitoring.

Build

Understand, create and inspect a Service Bus queue.

Extend

Choose an Azure AI service and explain a safe demo.

Next: optional appendix demos cover identity, operations and secure Azure website homework.

SQL: the Function needs an external database user

Already created in firstdb by an authorized Entra administrator

```
CREATE USER [fristAzureFunctionApp] FROM EXTERNAL PROVIDER;

GRANT SELECT, UPDATE ON OBJECT::lbs_training.Documents
    TO [fristAzureFunctionApp];
GRANT SELECT, UPDATE ON OBJECT::lbs_training.Outbox
    TO [fristAzureFunctionApp];
```

Enabling identity alone does not create this user.

The external user maps the workload identity; SQL grants authorize its table operations.

Who needs which permissions?

Use the narrowest scope; browser users and workload identities are different

Local website / developer

Azure CLI identity: SELECT + INSERT Documents; INSERT Outbox. Current lab login is an elevated admin.

Hosted Function identity

SELECT + UPDATE on Documents and Outbox. No INSERT, DELETE or db_owner is needed.

Hosted messaging

Data Sender + Data Receiver on document-processing. No broad subscription role is required.

Future Azure website

Separate web managed identity and SQL user. No Service Bus data role: it writes the outbox.

SQL network lesson: valid identity, blocked IP

Incident: processor invoked, but SQL rejected the connection

Observed failure

SQL logs: client IP 85.210.45.84 was not allowed. The document stayed Queued.

Why exact IP rules failed

Flex workers used changing addresses, including addresses absent from the reported list.

Owner's no-VNet lab choice

Enabled SQL: Allow Azure services and resources to access this server.

Security trade-off

Allows Azure-origin connection attempts from other subscriptions too. SQL identity and grants still apply.

“Allow Azure services” is not a universal trust rule

Network exceptions have different meanings in different Azure products

Azure SQL exception

Broad Azure-origin access; not limited to this Function, tenant, RG or subscription.

Service Bus trusted services

A service-specific supported list and scenarios. Not every Function is automatically exempt.

Storage trusted services

A separate Storage rule. It does not configure SQL or Service Bus access.

Lab boundaries

No VNet requested. No messaging/storage exception was changed to fix this SQL incident.

Hosted Function settings: identity-based connections

Double underscores separate configuration sections; preserve existing host storage

Sql__Server	firstsqlserver2026.database.windows.net
Sql__Database	firstdb
ServiceBusConnection__fullyQualifiedNamespace	fristazureservicebus.servicebus.windows.net
ServiceBusConnection__credential	managedidentity
QueueName	document-processing

Both Functions must be present: DispatchTrainingDocuments and ProcessTrainingDocument.

Testing: validate outcomes, not just delivery

Three Completed + seven Needs review for a fresh approved ten-case run

Expected sample results

01, 05, 06 → Completed. 02, 03, 04, 07, 08, 09, 10 → Needs review.

Offline checks

Test doubles cover validation, redelivery and failure paths. They do not prove Azure connectivity.

Live acceptance

Test-Live.ps1 -SubmitTrainingData writes ten real records and checks expected final statuses.

Persistence / content

Restart UI; records remain. Finalized SQL rows have CompletedAt and DocumentText NULL.

Troubleshooting and dead-letter recovery

Inspect first; fix the root cause before replaying approved identifiers

Look at the processor

ProcessTrainingDocument requests/exceptions. A healthy timer can dispatch zero while old jobs are dead-lettered.

Read the error layer

IP not allowed → network. Login failed → identity. Permission denied → SQL grants or messaging roles.

Inspect without consuming

Use Peek. Queue Active means enabled, not healthy; outgoing metrics include retries.

Replay safely

Approved IDs only: send replacement, confirm broker acceptance, then complete the locked DLQ message.

External attendees: guest identity is not RG access

Two steps: invite into Entra, then assign a scoped Azure resource role

Invite

Entra ID → Users → Invite external user. Guest accepts with their own account.

Assign

Training → IAM → Reader for observation. Contributor can change/delete resources; avoid Owner.

Verify scope

RG role covers that RG, not the whole subscription. Check inherited and group-based access.

Separate data permissions

SQL user + grants and queue Data Sender/Receiver are separate. Website users need app authorization.

Homework: host the website securely in Azure

Proposed enhancement only; current UI is still local and unauthenticated

Secure the users

Entra sign-in, approved-user restrictions and per-document authorization. Test denied access too.

Host the website

Approved App Service with supported .NET 10 configuration. Separate web managed identity.

Grant SQL only

Web identity: SELECT + INSERT Documents; INSERT Outbox. Reuse existing Functions and queue.

Operational follow-up

Ten-case regression, approved DLQ recovery, alerts, retention, cost review and rollback.